

Mutation (Instinct Evolution) Governed Through Verification, Routing, and Pinning: A Foundational Architectural Commitment in the Coordination Knowledge Substrate Pattern

Wenxin Li (Independent Researcher) — ORCID: 0009-0004-8065-3235

May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of three evolution mechanisms named in Paper 2’s Claim 4 — **mutation**, the undirected variation arising from LLM model evolution — as a standalone foundational architectural commitment with three governance instruments (verification, routing, pinning), instantiating the mechanism-specific governance shape Paper 2’s Claim 5 establishes.

Abstract

Paper 2 commits a CKS-governed AI Self to evolve through three mechanisms in productive tension under unified human governance: instinct evolution operating on the LLM and infrastructure layer, DNA evolution operating on the orchestration substrate, and action-feedback evolution closing the loop from recorded experience back into governed substrate refinement. The first — **instinct evolution**, also called **mutation** because it parallels biology’s undirected variation — is operationally distinctive: of the three, it is the only mechanism whose substrate content (the LLM’s internal behavior) the Self does not author or directly govern. CKS governs the *integration* of LLM evolution into the Self, not the LLM’s internal behavior. This note formalizes mutation as a standalone foundational commitment with three governance instruments — verification gates testing LLM behavior changes before integration, routing rules selecting which LLM version handles which consultation, and pinning rules routing high-stakes decisions to the reasoning layer. It states the commitment, distinguishes it from conventional unceremonious LLM updating, articulates the biological analog, traces inherited Paper 1 commitments, and bounds the limits.

1. Why mutation governance needs to be formalized as standalone foundational commitment

Paper 2’s Claim 4 names three evolution mechanisms; Claim 5 commits each to a distinct governance shape co-determined with mechanism shape. The integrating frame is itself formalized in B1.12. B1.13, B1.14, and B1.15 elaborate the three mechanisms in turn; this note, B1.13, formalizes mutation — the LLM-evolution mechanism — with its own governance machinery.

The foundational treatment matters for three reasons specific to mutation. First, mutation is the only mechanism whose substrate content arrives from outside the Self. The LLM is the vendor’s artifact, evolving on the vendor’s schedule under no Self-internal governance. Conventional AI deployments treat this asymmetry operationally — compatibility checks, prompt revisions, ad hoc evaluation. CKS treats it architecturally: LLM evolution is an *evolution event in the Self*, requiring governance even though the substrate evolving sits on the inside-the-model side of Paper 1’s substrate/LLM division. Second, the three instruments — verification, routing, pinning — serve distinct purposes a single composite framing would obscure. Third, mutation governance composes with several Paper 1 commitments (A1.01, A1.04, A1.05, A1.07, A1.10, A2.04) but does not reduce to any of them. Naming mutation governance as standalone makes the composition visible and the inherited-from-Paper-1 status precise.

2. The commitment, stated precisely

In the CKS pattern, **mutation** names the undirected variation arising from LLM model evolution: vendor model updates, training improvements, behavioral shifts in LLM responses over time, and analogous infrastructure-layer capability shifts arriving from upstream. The Self does not author this variation. Mutation is the biology-parallel undirected component of CKS’s evolutionary dynamics — the “random” component, in the sense that the Self does not choose what the next LLM version does differently from the prior one.

The commitment has two parts.

(a) Mutation governance is over integration, not over LLM weights. CKS makes no commitment about how the LLM was trained, what its weights are, or what its internal behavior should be. Those are the vendor’s. CKS commits to governance over how LLM evolution is *integrated* into the Self — when a new LLM version is adopted, for which consultations, with what verification record, under what routing rule, with what protections for high-stakes decisions.

(b) Integration governance operates through three instruments, each authored as orchestration content under the CKS rule-authoring discipline:

- **Verification.** Pre-integration testing. Before an LLM version is integrated, verification gates test whether it behaves as the deployment requires on substrate-resident test cases. Verification answers “does this version behave as we expect on the cases we care about.”
- **Routing.** Post-integration selection. Once a version has passed verification, routing rules determine which cell consultations use it. Different versions may handle different consultation types; multiple versions may operate concurrently. Routing answers “which consultations use this version.”
- **Pinning.** Architectural protection of high-stakes decisions. Pinning rules require designated decision categories — those whose consequences justify reasoning-layer processing rather than instinct alone — to be routed to the reasoning (substrate) layer where substrate-resident orchestration rules bound behavior independent of LLM version. Pinning answers “for which decisions do we not depend on instinct alone.”

Mutation events are recorded under the six-field provenance metadata Paper 1’s path retraceability commitment establishes, with the active LLM version identified in the provenance vocabulary. The governance machinery is itself substrate content under A1.01.

3. What makes mutation governance architecturally distinctive

Three distinctions follow.

Mutation is treated as architectural evolution event, not as operational integration task. Conventional AI deployments handle LLM updates as deployment-layer concerns — sometimes structured evaluation, often little more. CKS treats LLM updates as evolution events in the Self, with verification, routing, and pinning as architectural primitives co-determined with the mechanism — the same shape every mutation event takes, recorded as substrate content, under the inspect/modify/override rights A1.01 commits to.

Three instruments, distinct purposes, jointly necessary. Verification is pre-integration: before this version is allowed to be used, what does its behavior look like. Routing is post-integration: among the versions the Self has access to, which handles which consultation. Pinning is architectural: independent of LLM version, which decisions are required to go through the reasoning layer. A deployment with verification but no routing or pinning has the testing without the architectural protection; one with routing but no verification picks which untested version to use; one that pins but does not verify or route may protect high-stakes decisions while integrating arbitrary instinct behavior elsewhere. The three compose precisely because each carries operational content the others do not.

Mutation governance is what biology has no parallel for. Biology has no governance over mutation: variants arise stochastically and selection eliminates unfit ones over generations without architectural choice. CKS has no automatic selection filter for LLM behavior shifts; what it has instead is human-authored verification, routing, and pinning at the integration boundary — a class of governance biology has no analogue for, because biology has no vendor and no integration boundary.

4. The biological analog

The biology mimicry functions as conceptual scaffold rather than architectural commitment, as Paper 2 establishes for biology vocabulary throughout. Mutation parallels biological mutation in being undirected variation: in biology, mutation provides raw material for natural selection without itself selecting; in CKS, LLM evolution provides capability variation without itself selecting which variation a given Self adopts. The parallel is structural, not literal. Biology’s mutation is governance-free; CKS’s mutation is governance-bounded at the integration boundary — undirectedness in the variation arriving from upstream, directedness in the verification-routing-pinning machinery the Self holds at the integration boundary.

5. Inherited Paper 1 commitments

Mutation governance is not a new architectural axiom. It composes with several Paper 1 commitments and inherits their content directly.

A1.01 (human-governed: authority not labor). Verification, routing, and pinning rules are authored by humans under A1.01’s inspect/modify/override rights. The labor of executing them is allocable; authority over the rules is not. Mutation governance is the specific shape A1.01 takes when the substrate content evolving sits on the LLM side of Paper 1’s substrate/LLM division.

A1.04 (AI as substrate mediator). The mediator role is preserved through pinning. When a high-stakes decision is pinned to the reasoning layer, the substrate-mediator’s properties — LLM operating under substrate-resident rules, substrate as source of truth, humans holding override authority, provenance recorded — bound the behavior independent of LLM version.

A1.05 (tool-agnosticism). Tool-agnosticism makes routing operationally available. Because the substrate-host interface requires only persistent state, human read/write access, and LLM access to substrate content, the LLM end is replaceable. Which LLM version is invoked is a deployment choice tool-agnosticism makes governable.

A1.07 (path retraceability). Mutation events are recorded under the six-field provenance metadata: which

LLM version was active, when integration occurred, what verification result preceded it, what routing rule applied, what pinning rule held — auditable substrate content under the standard accountability vocabulary.

A1.10 (determinism contract). The contract is preserved via the bounded-non-determinism category Paper 1 recognizes. LLM behavior is non-deterministic, and the non-determinism varies across versions; the bounded category names this as recognized class, bounded by the requirement that the active LLM version be recorded in provenance.

A2.04 (orchestration rule authoring). Verification, routing, and pinning rules are authored as orchestration content under A2.04 — substrate content under the rule-authoring authority architecture, not deployment-time configuration.

What B1.13 names is the architectural shape these inherited commitments take when applied to mutation. None is new; the composition is.

6. Operational implications

Six implications follow at the deployment layer.

Verification suites are configured per deployment as substrate content. Test cases an LLM version must pass before integration are authored under A2.04 and held as substrate, reflecting consultation types, risk profile, and regulated obligations.

Routing rules are configured per consultation type. The same Self may operate with multiple LLM versions concurrently — one for low-stakes consultations where speed matters, one for high-stakes where the more capable version’s verification record is stronger, one for consultations requiring a specific behavioral profile.

Pinning rules are configured per decision category. Decisions carrying regulatory consequence, irreversibility, safety implication, or other high-stakes property are enumerated as pinning rules and routed to substrate-resident orchestration regardless of how capable the available instinct has become. As instinct sharpens, pinning rules may be revised under governance.

Vendor changes trigger re-verification and re-evaluation. When an LLM vendor releases a new model version, or a different vendor must be substituted, verification suites are re-run, routing rules re-evaluated, and pinning rules reviewed — a substrate-level event with full provenance, not an ad hoc action.

Multi-version operation is supported as primary deployment mode. Mutation governance does not assume one-LLM-per-deployment; it commits to versioned LLM use under routing as architecturally available.

Regulated work emphasizes pinning. In settings where decisions cannot rely on instinct alone — clinical care, regulated finance, safety-critical operations — pinning is the architectural primitive that protects the requirement. Substrate-resident reasoning rules bound decision behavior independent of LLM version, allowing the deployment to satisfy the regulatory commitment without depending on the vendor’s behavior remaining stable.

7. Limits

The standalone foundational treatment of mutation governance is bounded in six ways.

The commitment does not govern LLM weights. CKS makes no claim about training, fine-tuning, alignment work, or any other aspect of the LLM’s internal behavior. Those are the vendor’s.

The commitment does not eliminate LLM behavior shifts. Mutation governance handles such shifts architecturally; it does not prevent them. The architecture’s response is to make such cases governance-

visible and bounded by pinning where consequences justify it.

Verification is not exhaustive. Verification suites cover what they are authored to cover. The commitment is to make verification a governance event with substrate-resident records, not that any particular suite is complete.

Routing is not automatic. Routing rules are authored under A2.04 and reflect deployment choices; they are not derived from observed behavior or auto-tuned by optimization.

Pinning does not eliminate instinct use. Pinning bounds where instinct alone is insufficient; it does not require all decisions be reasoning-layer-only. The instinct/reasoning separation B1.01 establishes is preserved as a working architecture.

Mutation governance does not replace the other two mechanisms. Each has its own scope per B1.12; DNA evolution and action-feedback evolution are governed under their own shapes (B1.14, B1.15). The three together constitute Paper 2’s evolution architecture.

A scoping point: mutation governance is operationally distinct from forced vendor migration under vendor-unavailability boundary cases. Forced migration is reactive; mutation governance is proactive. The two interact — forced migration triggers mutation governance for the substituted version — but they are architecturally distinct events.

8. Operational test

A system instantiates the CKS mutation governance commitment if and only if all of the following are true at all times during the substrate’s existence:

1. LLM version integrations are governance events recorded as substrate content under the six-field provenance metadata A1.07 commits to, identifying the version, time of integration, verification record, and applicable routing rule.
2. Verification rules, routing rules, and pinning rules are authored under A2.04 as orchestration content, held as substrate, inspectable and modifiable under A1.01.
3. The system can operate with multiple LLM versions concurrently, with routing rules selecting which version handles which consultation.
4. Designated decision categories are pinned to reasoning-layer processing per pinning rules, routed through the substrate-resident orchestration A1.04 specifies, independent of which LLM version is otherwise active.
5. Mutation-related variability is recognized as a bounded non-determinism category per A1.10, with the active LLM version recorded in provenance.

A system that fails any of (1)–(5) may be a useful system that integrates LLM updates competently in some other framework, but does not instantiate mutation governance in the CKS sense.

9. Why naming as standalone matters; position in the evolution cluster

Mutation is the first of three evolution mechanisms B1.13–B1.15 elaborate under the integrating frame B1.12 establishes. Naming it as standalone commitment with its own three-instrument governance machinery does three things the integrating frame alone could not.

It distinguishes mutation governance from the other mechanisms’ governance shapes precisely. B1.14 will formalize directed selection (DNA evolution) under the standard authority architecture from Paper 1; B1.15 will formalize action-feedback evolution under proposal-and-acceptance machinery. Verification machinery for upstream-arriving capability shifts cannot work for proposed substrate changes, and authority architecture

for proposed substrate changes cannot work for action-feedback approval.

It surfaces the integration-boundary commitment mutation governance specifically carries. Of the three mechanisms, only mutation operates over substrate content the Self does not author. The integration-boundary commitment — that CKS governs integration, not the substrate’s internal behavior — distinguishes mutation governance from the other two.

It provides the foundational basis on which subsequent Phase B1 notes build. B1.14 and B1.15 elaborate the other two mechanisms; B1.16 develops bidirectional evolution; B1.17–B1.20 develop structural properties of the multi-mechanism evolutionary architecture.

Subsequent work that implements, extends, or argues against this commitment should use “mutation,” “verification,” “routing,” and “pinning” in the senses formalized here. Work using these terms differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Mutation (Instinct Evolution) Governed Through Verification, Routing, and Pinning: A Foundational Architectural Commitment in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.